

Smart Bank Management System (SBMS)

Programming Fundamentals – Project Report

Course: IN1101 | June 2026

Group Members: 254100M – Jinusha P. | 254101R – Kalansooriya O.K. | 254102V – Kalpanee K.A.C. | 254103B – Kan M.J.Z.

1. Introduction

The Operational Challenge. Running the everyday operations of a bank – keeping customer records up to date, opening and maintaining accounts, processing deposits and withdrawals, and managing loans – becomes difficult to coordinate when the information lives in disconnected paper files or scattered spreadsheets. Records get misfiled, account balances drift out of sync with the transactions that produced them, history is lost, and loan applications stall because their status is never tracked in one place. The Smart Bank Management System (SBMS) tackles this by consolidating all four areas into a single, menu-driven console application where customers, accounts, transactions, and loans are linked together and managed consistently.

2. Short Project Report

Core Features & Mechanics

- **Customer & Account Management:** The system manages the full lifecycle of customers and accounts – adding, viewing, searching, updating, and deleting each – and ties every account back to its owner through a shared Customer ID.
- **Real-Time Transaction Processing:** Deposits and withdrawals immediately adjust the relevant account balance, and every movement is written to a transaction log with a unique transaction ID so a complete, searchable audit trail is always available.
- **Loan Lifecycle Management:** Loan applications progress through a controlled status workflow – Pending, Approved, then Paid – with installment repayments steadily reducing the outstanding balance until the loan is settled.
- **Menu-Driven Navigation:** A central main menu routes the user to four module sub-menus, each offering five operations, and cleanly returns control to the menu after every action so the program never dead-ends.

3. Group Members and Individual Contributions

- **254100M – JINUSHA P. (Customer Management):** Built the customer module – adding new customers, viewing and searching the customer list, and updating or deleting existing records – and designed the Customer structure that the rest of the system references.
- **254101R – KALANSOORIYA O.K. (Account Management):** Developed account creation, viewing, searching, updating, and deletion, and implemented the Customer ID mapping that links each account to the customer who owns it.
- **254102V – KALPANE K.A.C. (Transaction Management):** Implemented deposits and withdrawals with live balance updates, the global transaction history log, transaction search, and the option to clear the history.

- **254103B – KAN M.J.Z. (Loan Management):** Created the loan application process, loan search, the approval workflow built around a status field, installment repayment handling, and loan status reporting.

4. System Overview

The Smart Bank Management System is a console-based application written in the C programming language that simulates real-world banking operations. It is organised around a central main menu that directs the user to four functional modules – Customer, Account, Transaction, and Loan. Each module stores its data in an array of structures with capacity for fifty records, and the modules share information through Customer ID and Account Number references, so an account always knows which customer it belongs to and a transaction always knows which account it affects. Sample data is initialised when the program starts, allowing every feature to be demonstrated immediately without manual setup.

Data Structures Used

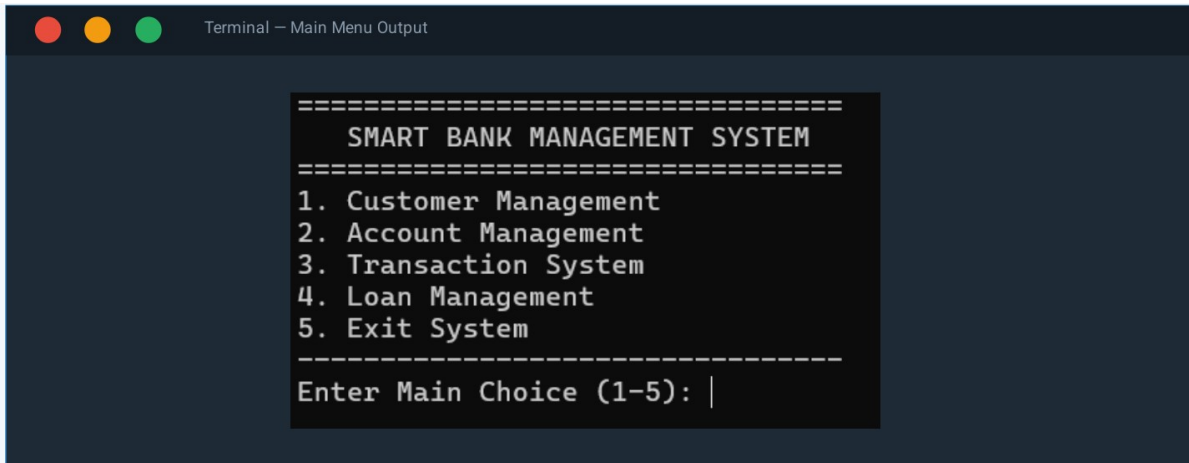
Four C structures form the data backbone of the system. Each models one core banking entity and is stored in its own fixed-size array:

Structure	Member Fields
struct Customer	customerID (int), name (char[50]), phone (char[15]), email (char[50])
struct Account	accountNo (int), customerID (int), accountType (char[20]), balance (double)
struct Transaction	transactionID (int), accountNo (int), type (char[20]), amount (double)
struct Loan	loanID (int), customerID (int), loanAmount (double), balanceAmount (double), status (char[20])

Programming Concepts Used

- **Structures:** Four structures (Customer, Account, Transaction, and Loan) model the core entities the bank works with.
- **Arrays of Structures:** Fixed-size arrays hold up to fifty records of each entity type, with counters tracking how many are currently in use.
- **Functions & Modular Programming:** Each operation is written as its own function and grouped into self-contained modules, keeping the code organised and easy to maintain.
- **Menu-Driven Programming & Switch-Case:** Nested menus driven by switch-case statements route the user from the main menu down to individual operations.
- **Loops & Conditional Statements:** Loops iterate over stored records while if-else checks guard each operation before it runs.
- **Searching Techniques:** Linear search locates records by Customer ID, Account Number, or Loan ID.
- **Global Variables:** Shared record arrays and counters preserve the system state across all functions.
- **Data Validation:** Input checks reject invalid amounts, missing records, and malformed entries before any change is committed.

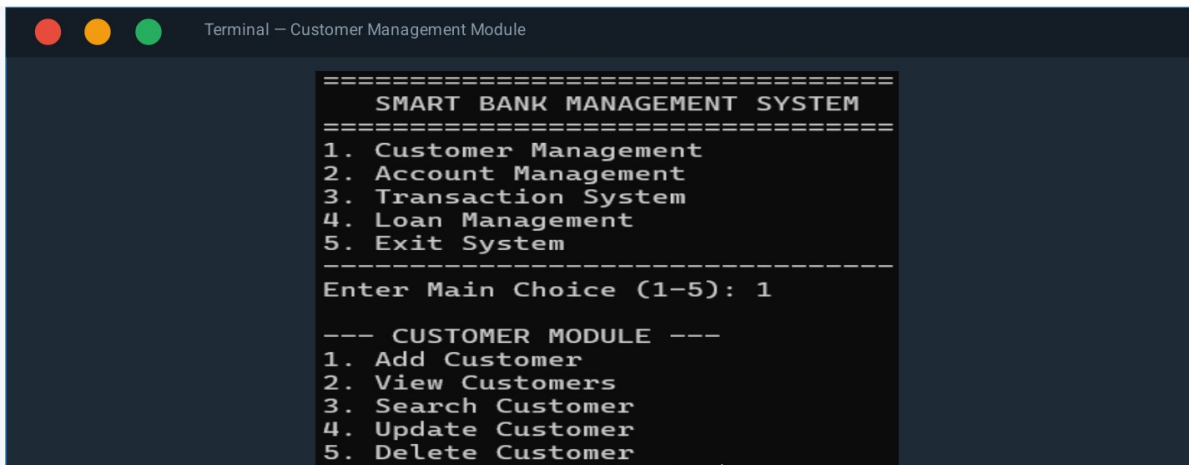
5. Screenshots of Outputs



```
Terminal - Main Menu Output

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
-----
Enter Main Choice (1-5): |
```

Main Menu

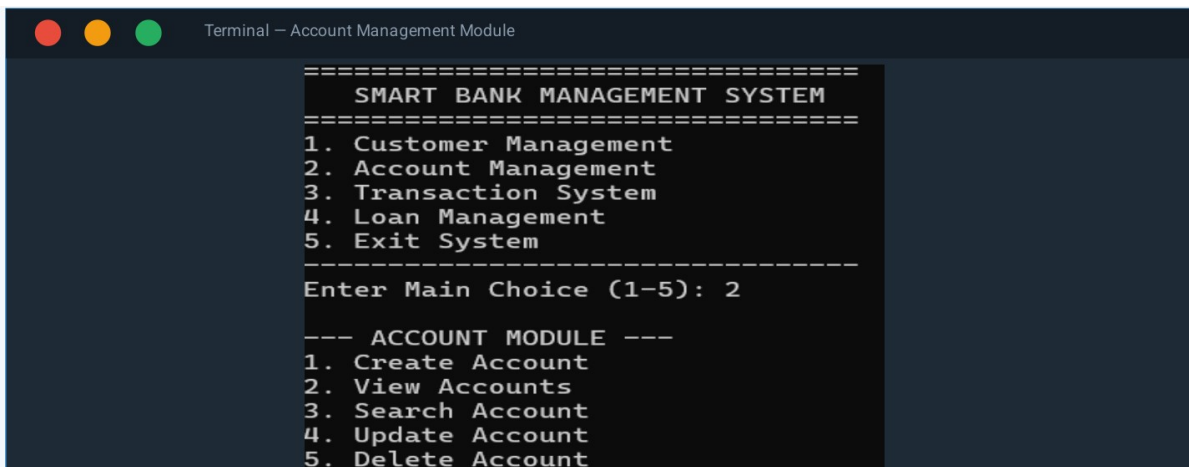


```
Terminal - Customer Management Module

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
-----
Enter Main Choice (1-5): 1

--- CUSTOMER MODULE ---
1. Add Customer
2. View Customers
3. Search Customer
4. Update Customer
5. Delete Customer
```

Customer Management Module



```
Terminal - Account Management Module

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
-----
Enter Main Choice (1-5): 2

--- ACCOUNT MODULE ---
1. Create Account
2. View Accounts
3. Search Account
4. Update Account
5. Delete Account
```

Account Management Module

```
Terminal — Transaction Management Module

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
-----
Enter Main Choice (1-5): 3

--- TRANSACTION MODULE ---
1. Deposit Money
2. Withdraw Money
3. View Transaction History
4. Search Transactions
5. Clear All History
```

Transaction Management Module

```
Terminal — Loan Management Module

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
-----
Enter Main Choice (1-5): 4

--- LOAN MODULE ---
1. Apply for Loan
2. View All Loans
3. Search Loan
4. Approve Loan
5. Pay Loan Installment
```

Loan Management Module

```
Terminal — Transaction History Output

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
-----
Enter Main Choice (1-5): 3

--- TRANSACTION MODULE ---
1. Deposit Money
2. Withdraw Money
3. View Transaction History
4. Search Transactions
5. Clear All History
Enter Operation (1-5): 1

Enter Account Number for Cash Deposit: 501
Enter Deposit Amount: 1000

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
-----
Enter Main Choice (1-5): 3

--- TRANSACTION MODULE ---
1. Deposit Money
2. Withdraw Money
3. View Transaction History
4. Search Transactions
5. Clear All History
Enter Operation (1-5): 3

--- Global Core Transaction History Logs ---
TX ID      Account No.  Type      Amount
```

Deposit & Transaction History Output

```
Terminal — Loan Approval & Payment Output

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
=====
Enter Main Choice (1-5): 4

--- LOAN MODULE ---
1. Apply for Loan
2. View All Loans
3. Search Loan
4. Approve Loan
5. Pay Loan Installment
Enter Operation (1-5): 1
Enter Customer ID applying for loan: 101
Enter Requested Loan Principal Amount: 50000
Application filed! Tracking Reference Loan ID: 201

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
=====
Enter Main Choice (1-5): 4

--- LOAN MODULE ---
1. Apply for Loan
2. View All Loans
3. Search Loan
4. Approve Loan
5. Pay Loan Installment
Enter Operation (1-5): 4
Enter Loan ID reference code for underwriting decision: 201
Success! Loan ID 201 tracking flags marked as APPROVED.

=====
SMART BANK MANAGEMENT SYSTEM
=====
1. Customer Management
2. Account Management
3. Transaction System
4. Loan Management
5. Exit System
=====
Enter Main Choice (1-5): 4

--- LOAN MODULE ---
1. Apply for Loan
2. View All Loans
3. Search Loan
4. Approve Loan
5. Pay Loan Installment
Enter Operation (1-5): 5
Enter Loan ID profile reference for settlement payment: 201
Current Remaining Obligation Debt Balance: 50000.00
Enter Your Repayment Payment Amount: 5000
Installment Payment Posted! Balance Due Left: 45000.00 LMR
```

Loan Application, Approval & Repayment Workflow

6. Challenges Faced and Solutions

Challenge 1: Coordinating Multiple Modules. Combining four independent areas – customers, accounts, transactions, and loans – into one program risked tangled, hard-to-follow code where one module could interfere with another.

Solution: We split every operation into its own function and kept each module self-contained, with a single main menu acting as the router. This modular structure made the program easier to build, test, and debug as a team.

Challenge 2: Maintaining Customer–Account Relationships. An account has no meaning on its own; it must always be tied to the correct customer, and a naive design could easily lose that link.

Solution: We stored a Customer ID inside the Account structure and cross-referenced the customer array whenever account details were displayed, guaranteeing every account stays connected to its rightful owner.

Challenge 3: Tracking Transaction Data Reliably. Every deposit and withdrawal needed to be recorded accurately and remain searchable later, which is impossible if transactions are not stored in a structured way.

Solution: We introduced a dedicated Transaction structure, giving each movement a unique transaction ID and account reference, and appended it to a global history log that can be viewed and searched at any time.

Challenge 4: Modelling the Loan Approval Workflow. A loan is not a single event but a process, and treating it as a one-off entry made it impossible to tell which loans were still pending, approved, or fully repaid.

Solution: We added a status field that moves through Pending, Approved, and Paid. The status gates what actions are allowed, so a loan cannot be repaid before it is approved, mirroring a real bank's workflow.

Challenge 5: Storing Records Without a Database. Without access to a database, the system still needed a dependable way to hold many records in memory throughout the session.

Solution: We used fixed-size arrays of structures – a maximum of fifty records per entity – paired with counters that track exactly how many slots are filled, preventing out-of-range access.

Challenge 6: Preventing Invalid Input. Users could enter negative amounts, reference accounts that do not exist, or attempt to withdraw more than an account holds, any of which would corrupt the data.

Solution: We placed logical condition checks before every operation – confirming the record exists, the amount is positive, and the balance is sufficient – so invalid actions are rejected with a clear message rather than silently breaking the system.

7. Future Enhancements

While the current system fully demonstrates the core banking workflow, several enhancements would move it closer to a production-grade application:

- **File Handling:** Saving records to disk so data persists permanently across sessions instead of resetting when the program closes.
- **User Authentication:** A secure login system with role-based access for staff and administrators.
- **Interest Calculation:** Automated computation of interest on loans and savings accounts.
- **ATM Simulation:** A virtual ATM interface module for customer-facing self-service operations.
- **Database Integration:** Replacing the fixed-size arrays with an SQL or NoSQL backend to remove the record limit.
- **Mobile Banking App:** A cross-platform mobile interface built on top of the same core logic.

8. Conclusion

The Smart Bank Management System successfully brings the core operations of a bank – customer management, account handling, financial transactions, and loan processing – together in a single, well-organised C application. By relying on structures, arrays of structures, modular functions, and disciplined input validation, the system stays robust and predictable even as records are added, updated, and removed. Beyond the working software, the project strengthened the team's problem-solving, collaboration, and understanding of how individual modules combine to solve a larger, realistic problem.

GitHub Repository: [*Insert your GitHub repository link here*]